the behavior of an individual class, component, or API. In both cases the approach is similar; only the audience and the toolset change.

For technical living documentation, sophisticated reporting capabilities are less important than code readability and clarity. The primary audience of technical documentation is the developer who will need to maintain the code later on. Developers are used to reading code, so technical documentation in the form of well-organized, readable, annotated code samples is generally quite sufficient to help them understand the finer points of the code base. Many teams complement the low-level living documentation with light, higher-level architectural documentation, stored, for example, on a project wiki.

### 11.6.1 Unit tests as living documentation

Some tools make it a little more natural to express unit tests as living documentation. Spock, for example, is a great way to write low-level BDD tests for Java or Groovy projects. As you saw in chapter 10 (section 10.4.3), a simple Spock specification might look like this:

A short text explains what each step is doing.

```
def "Find flight details by flight number"() {
    given: "I need to know the details of flight number FH-101"
        def flightService = new FrequentFlyerFlightService();
        def flightNumber = "FH-101"
        def airport = "MEL"
    when: "I request the details about this flight"
        def flightDetails = flightService.findFlightByNumber(airport,
                                                             flightNumber);
    then: "I should receive the correct flight details"
        flightDetails.flightNumber == "FH-101" &&
        flightDetails.departure.name == "Melbourne" &&
        flightDetails.destination.name == "Sydney" &&
        flightDetails.time == "06:00"
}
```

But writing unit tests that make good technical documentation relies more on an attitude than on using a particular tool. The following NSpec specification, for example, also does a great job of explaining what feature it's describing and illustrating how an API should be used:

```
public class WhenUpdatingStatusPoints : nspec
{
    FrequentFlyer member;

    void before_each()
    {
        member = new FrequentFlyer();
    }

    void earning_status_points()
    {
       context["When cumulating Frequent Flyer points"] = () =>
```